Logistic Regression

In [ ]:
```
#Step1:importing
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score,confusion_matrix,classification_r
```

In [2]:
```
df = pd.read_csv('diabetes.csv')
df.head()
```

Out[2]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunc
0	6	148	72	35	0	33.6	0.627
1	1	85	66	29	0	26.6	0.351
2	8	183	64	0	0	23.3	0.672
3	1	89	66	23	94	28.1	0.167
4	0	137	40	35	168	43.1	2.288

In [3]:
```
df.shape
```

Out[3]:
```
(768, 9)
```

In [4]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness             768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64
 8   Outcome                   768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
```

In [4]:
```
df['Outcome']=df['Outcome'].astype('category')
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness             768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64
 8   Outcome                   768 non-null    category
dtypes: category(1), float64(2), int64(6)
memory usage: 49.0 KB
```

In [5]:
```
df.isnull().sum()
```

Out[5]:
```
Pregnancies                 0
Glucose                     0
BloodPressure               0
SkinThickness               0
Insulin                     0
BMI                         0
DiabetesPedigreeFunction    0
Age                         0
Outcome                     0
dtype: int64
```

In [6]:
```
x= df.drop('Outcome', axis=1)
#x = df.iloc[:,:-1]#input
y=df['Outcome']#target
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.3,random_stat
```

In [7]:
```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
x_train = scaler.fit_transform(x_train)
x_test = scaler.fit_transform(x_test)
```

In [8]:
```

cl=LogisticRegression()
```

In [9]:
```
cl.fit(x_train,y_train)
```

Out[9]:
```
LogisticRegression()
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

In [12]:
```
x_test
```

Out[12]:
```
array([[ 0.88223199,  1.81182773,  0.7454402 , ...,  0.4822581 ,
         0.28279261,  1.55940276],
       [-0.55294822,  0.14817377, -0.52755208, ..., -0.47337878,
         3.34632252, -0.69306789],
       [ 2.03037615,  0.48706624,  0.35375027, ...,  0.16759718,
        -0.21873201,  0.17326697],
       ...,
       [-1.1270203 ,  0.08655695,  1.33297511, ..., -1.07939241,
        -0.69608677, -1.03960184],
       [-0.83998426, -0.03667667,  0.45167275, ...,  0.84353546,
        -0.699108  , -0.43316743],
       [ 0.3081599 , -0.43718596,  0.1579053 , ...,  0.50556632,
        -0.69306554,  0.        ]])
```

In [10]:
```
y_pred=cl.predict(x_test)
y_pred
```

Out[10]:
```
array([1, 0, 1, 0, 0, 1, 1, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0,
       0, 1, 1, 0, 0, 0, 0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1, 0, 0, 1, 0, 0, 0,
       1, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1, 0,
       1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0,
       1, 0, 1, 1, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1,
       1, 0, 0, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0,
       0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0,
       0, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0,
       1, 0, 0, 0, 0, 1, 1, 0, 1, 1, 0, 1, 0, 1, 0, 1, 0, 0, 1, 0, 1, 1,
       1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0], dtype=int64)
```

In [11]:
```
#probality
probs = cl.predict_proba(x_test)
probs
```

Out[11]:
```
array([[0.16684784, 0.83315216],
       [0.65675842, 0.34324158],
       [0.38587446, 0.61412554],
       [0.94531983, 0.05468017],
       [0.9303394 , 0.0696606 ],
       [0.383763  , 0.616237  ],
       [0.09981623, 0.90018377],
       [0.53623759, 0.46376241],
       [0.41234002, 0.58765998],
       [0.70428937, 0.29571063],
       [0.1560307 , 0.8439693 ],
       [0.82407886, 0.17592114],
       [0.86859264, 0.13140736],
       [0.7665155 , 0.2334845 ],
       [0.17893132, 0.82106868],
       [0.95654811, 0.04345189],
       [0.75664163, 0.24335837],
       [0.92740945, 0.07259055],
       [0.94193892, 0.05806108],
```

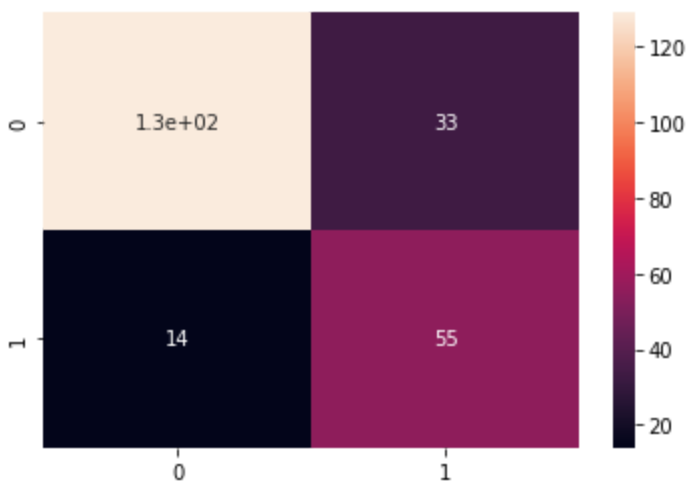
In [20]:
```
print("Accuracy :",accuracy_score(y_pred,y_test))
print("cls_Report :\n",classification_report(y_pred,y_test))
confusion_matrix(y_pred,y_test)
```

```
Accuracy : 0.7965367965367965
cls_Report :
               precision    recall  f1-score   support

           0       0.90      0.80      0.85       162
           1       0.62      0.80      0.70        69

    accuracy                           0.80       231
   macro avg       0.76      0.80      0.77       231
weighted avg       0.82      0.80      0.80       231
```

Out[20]:
```
array([[129,  33],
       [ 14,  55]], dtype=int64)
```

In [21]:
```
sns.heatmap(confusion_matrix(y_pred,y_test),annot=True)
```

Out[21]:
```
<AxesSubplot:>
```



In [17]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness             768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64
 8   Outcome                   768 non-null    category
dtypes: category(1), float64(2), int64(6)
memory usage: 49.0 KB
```

In [19]:
```
X = [[2, 120, 70, 25, 80, 28.5, 0.5, 33]]  # custom input
X_scaled = scaler.fit_transform(X)
res=cl.predict(X_scaled)[0]
if(res==0):
    print("No :) Diabetes for you...")
else:
    print("Sorry :( you have diabetes")
res
```

```
No :) Diabetes for you...
```

Out[19]:
```
0
```